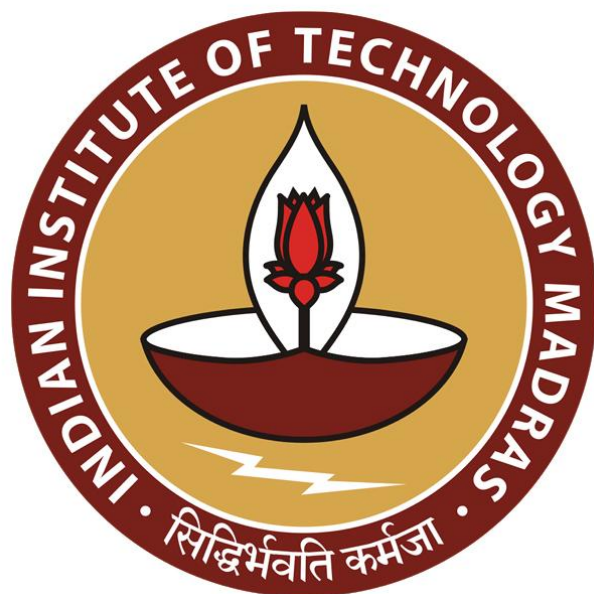




**IIT Madras**  
ONLINE DEGREE

**Programming Concepts using Java**  
**Professor Madhavan Mukund**  
**Department of Computer Science Chennai Mathematical Institute**  
**Indian Institute of Technology, Madras**  
**Polymorphism Revisited**

(Refer Slide Time: 00:13)

The slide features a blue header with the text "Polymorphism revisited". Below the header, the name "Madhavan Mukund" and the URL "https://www.cmi.ac.in/~madhavan" are displayed. The main content area shows a video feed of the professor, who is wearing a blue and white checkered shirt. The slide also includes the text "Programming Concepts using Java" and "Week 5".

So, let us take a closer look at this word polymorphism, which we have come across earlier.

(Refer Slide Time: 00:19)

The slide is titled "Polymorphism" and explains the concept of dynamic dispatch. It includes a list of bullet points:

- In object-oriented programming, **polymorphism** usually refers to the effect of dynamic dispatch
- **S** is a subclass of **T**
- **S** overrides a method **f()** defined in **T**
- Variable **v** of type **T** is assigned to an object of type **S**
- **v.f()** uses the definition of **f()** from **S** rather than **T**

A diagram illustrates the concept of dynamic dispatch. It shows a variable **v** of type **T** (represented as **Tv = o**) pointing to an object of type **S**. The object **S** has a method **f()** that overrides the method **f()** defined in the superclass **T**. The diagram shows that when **v.f()** is called, it uses the definition of **f()** from **S** rather than **T**.

## Polymorphism

- In object-oriented programming, **polymorphism** usually refers to the effect of dynamic dispatch
  - S** is a subclass of **T**
  - S** overrides a method **f()** defined in **T**
  - Variable **v** of type **T** is assigned to an object of type **S**
  - v.f()** uses the definition of **f()** from **S** rather than **T**
- Every object "knows" what it needs to do

Simple



Madhavan Mukund

Polymorphism revisited

Programming Concepts using Java 2 / 6

## Polymorphism

- In object-oriented programming, **polymorphism** usually refers to the effect of dynamic dispatch
  - S** is a subclass of **T**
  - S** overrides a method **f()** defined in **T**
  - Variable **v** of type **T** is assigned to an object of type **S**
  - v.f()** uses the definition of **f()** from **S** rather than **T**
- Every object "knows" what it needs to do
- More generally, polymorphism refers to behaviour that depends only a specific capabilities — **structural polymorphism**
  - Reverse an array/list (should work for any type)
  - Search for an element in an array/list
  - Sort an array/list



Madhavan Mukund

Polymorphism revisited

Programming Concepts using Java 2 / 6

सिद्धिर्भवति कर्मजा



## Polymorphism

- In object-oriented programming, **polymorphism** usually refers to the effect of dynamic dispatch
  - **S** is a subclass of **T**
  - **S** overrides a method **f()** defined in **T**
  - Variable **v** of type **T** is assigned to an object of type **S**
  - **v.f()** uses the definition of **f()** from **S** rather than **T**
- Every object "knows" what it needs to do
- More generally, polymorphism refers to behaviour that depends only a specific capabilities — **structural polymorphism**
  - Reverse an array/list (should work for any type)
  - Search for an element in an array/list (need equality check)
  - Sort an array/list



Madhavan Mukund

Polymorphism revisited

Programming Concepts using Java 2 / 6



## Polymorphism

- In object-oriented programming, **polymorphism** usually refers to the effect of dynamic dispatch
  - **S** is a subclass of **T**
  - **S** overrides a method **f()** defined in **T**
  - Variable **v** of type **T** is assigned to an object of type **S**
  - **v.f()** uses the definition of **f()** from **S** rather than **T**
- Every object "knows" what it needs to do
- More generally, polymorphism refers to behaviour that depends only a specific capabilities — **structural polymorphism**
  - Reverse an array/list (should work for any type)
  - Search for an element in an array/list (need equality check)
  - Sort an array/list (need to compare values)



Madhavan Mukund

Polymorphism revisited

Programming Concepts using Java 2 / 6

So, in object-oriented programming, as we have seen it so far, we use the word polymorphism to usually refer to what we normally achieved through what is called dynamic dispatch. So, remember, the dynamic dispatch says that the function that an object executes depends on its runtime type and not on its declared or assigned type at compile time.

So, in particular, if you have two classes, so you have **S**, which is subclass of **T**, so that means an object of type **S** can do everything that an object can type **T** can do and more. And in particular, if you have a variable of type **T**, you can assign it an object of type **S**. So, wherever you expect a **T**, you are willing to take an **S**, because **S** can do everything that **T** can do and more. Now, **T**



might define a function `f`. So, `T` is here, and it defines a function `f`, and `S` is a subtype and it defines an alternative implementation of `f`, which overrides the method `f` in `T`. So, `S` has a new implementation of `f`.

So, now, as we said, if you have a declaration of this type, capital `T`, `v`, which says that `v` is of type `T`, I can now make this point to an object of type `S`. So, I can assign `v` an object of type `S`. And now when I call `f` on this object, the question is which `f` are we going to use, because there is a compile time `f`, which is associated with the type of `v`, and there is an object here who is of type capital `S`, and so it has its own more specialized definition of `f`. And what dynamic dispatch says is that the runtime value is the one that is chosen. So, in this particular case, because the object itself his identity at runtime is of type `S` and not `T`, he will use the function of `f`, which comes with `S` and not with `T`.

So, this is useful when we have a kind of heterogeneous mix. So, remember, the original simulate example where we had a queue of events and we wanted to associate with each event a simulate event, action. So, when we pick up an event from this queue and we say simulate, it knows how to simulate itself. So, we do not have to say if it is of this type to this, if it is of that type, so we avoid this kind of elaborate if else case structure to try to distinguish the type and choose the appropriate function to involve, to invoke on it, rather we assume that these are all somehow compatible types.

They are bound together by having a common super type. And each of them has this function simulate. And therefore, if I say `e.simulate()`, then it will work regardless of what type of `e` it is. So, every object knows what it needs to do. So, based on its dynamic identity, it will appropriately behave. So, we can have this array or list or queue of objects of compatible but different types and just invoke the same function on each of them and each of them will know what to do.

So, this is a very limited view of polymorphism which is associated with having one type overriding the function of another type. Now, in, normally in programming languages, polymorphism is more general than this. And it is talking about behaviors that require only

specific capabilities. So, we have already seen this in a sense and sorting, but let us look at three examples.

The first is supposing I have an array or a list of objects and I want to reverse it, now, clearly, what we are saying is that this reversing a list or reversing an array is something to do with the positions of the elements, it does not depend on the elements themselves. So, if I put you in a room with a bunch of boxes and I say reverse the order of the boxes, you do not need to know what is inside the box. So, that is a good way of thinking about polymorphism. I have some boxes and do I need to look inside the box to what extent do I need to look inside the box. So, this is one example. I need to take a list for a sequence of values and reverse them.

Slightly different is one which we saw before kind of find function which takes a sequence an array of objects and searches for one object and the other. And the third one is one where we actually want to arrange this array or list in a particular order based on some quantity. So, now in all of these cases, there is a limited amount of information that we need about the objects in order to do this. So, this is sometimes if you want to distinguish it from this inheritance polymorphism as we find in object-oriented programming, because there is this confusion in programming languages about what polymorphism refers to because the two communities of, typically of functional programming types and object-oriented programming types use it in different ways.

So, so far what we have seen is this inheritance type of polymorphism, which comes from this class hierarchy. What we are talking about here is a more functional aspect. We want to do something with some objects, what do we need to know about them. So, this is sometimes called structural polymorphism, just to distinguish it. So, if somebody just says polymorphism, we need to be clear which one of them they are talking about, because unfortunately it is an ambiguous term.

So, as we said, if I just want to reverse the positions of objects in a list, imagine you are in this room with a bunch of cartons and you are just asked to rearrange them in the opposite order, you do not need to open the cartons. So, this should work for any time. How about this one, searching for an element in an array or a list? Again, this seems that it should work for any type. So, if I give you a bunch of objects and I asked you to search for something, it would work for any type

of objects. But there is a small subtlety, because I need to be able to check whether or not the object I am looking for is in that array.

So, there is something which is implicit which we need to make explicit, which is we need to be able to compare the object we have on the in our hand with the ones there. For example, if I give you an empty carton and I asked you whether it is the same as one of the cartons in the room, now, I have to specify what I mean by is the same, is the external appearance of the carton the same that you can certainly check. But if I want you to check the insides of the carton, then you will have to open the carton. So, you need more than just the object itself. So, you need an equality check. So, this requires an extract capability.

And of course, sorting we have already seen before, in order to sort something, you need to be able to compare two elements and decide which one is smaller, which one is bigger. So, you need a comparison function, not just whether one is equal to the other or not. So, just to emphasize why this equality check is not trivial, imagine that you have some capability of having functions in a list. So, I could have a list of sort functions. I can have quicksort. I can have merge sort. I can have selection sort. And now I asked you whether a given piece of code that I have written is one of these. Now, how would you check that?

You will now have to check whether a function that I have written is the same as the function which is there in that list. Now, it could be rewritten in a different format so the algorithm could be the same. It could be identical. So, of course, if it is textually identical, it is easy. But if it is written in a different way, maybe with different variables and different indentation, then it becomes a problem. And if you want functional equality, then of course, all the sorting algorithms up to a certain thing are correct, or correct sorting algorithms are all the same, they all sort, but the functional dependency could depend on the speed.

So, this notion of equality, I just want to emphasize, is not something which is obvious. It might be straightforward for values which have a kind of simple type, but if they are objects or if they are more complicated things, then the notion of equality may not even make sense. In fact, one of the fundamental results, as we talked about computing is that when we talked about, for example,

type checking and all that is we cannot check if two programs are equal if they do the same thing. So, therefore, equality at the level of programs is something we certainly cannot do.

(Refer Slide Time: 08:12)

IIT Madras  
BSc Degree

## Structural polymorphism

- Use the Java class hierarchy to simulate this



Madhavan Mukund

Polymorphism revisited

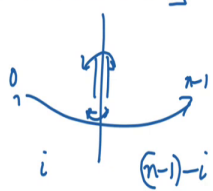
Programming Concepts using Java 3 / 6

IIT Madras  
BSc Degree

## Structural polymorphism

- Use the Java class hierarchy to simulate this
- Polymorphic `reverse`

```
public void reverse (Object[] objarr){  
    Object tempobj;  
    int len = objarr.length;  
    for (i = 0; i < n/2; i++){  
        tempobj = objarr[i];  
        objarr[i] = objarr[(n-1)-i];  
        objarr[(n-1)-i] = tempobj;  
    }  
}
```



Madhavan Mukund

Polymorphism revisited

Programming Concepts using Java 3 / 6

## Structural polymorphism

- Use the Java class hierarchy to simulate this

- Polymorphic **reverse**

- Polymorphic **find**

- == translates to Object.equals()

```
public int find (Object[] objarr, Object o){
    int i;
    for (i = 0; i < objarr.length; i++){
        if (objarr[i] == o) {return i;}
    }
    return (-1);
}
```



Madhavan Mukund

Polymorphism revisited

Programming Concepts using Java 3 / 6

## Structural polymorphism

- Use the Java class hierarchy to simulate this

- Polymorphic **reverse**

- Polymorphic **find**

- == translates to Object.equals()

- Polymorphic **sort**

- Use interfaces to capture capabilities

```
public interface Comparable{
    public abstract int cmp(Comparable s);
}

public class SortFunctions{
    public static void quicksort(Comparable[] a){
        ...
        // Usual code for quicksort, except that
        // to compare a[i] and a[j] we use
        // a[i].cmp(a[j])
    }
}
```



Madhavan Mukund

Polymorphism revisited

Programming Concepts using Java 3 / 6

So, if we are trying to use Java to implement this notion of structural polymorphism, then what we have seen is that we can use this Java class hierarchy, use the fact that object is a kind of generic super type of everything. So, we have this tree like structure of all classes. And at the top, we have this object class. So, object is ancestor of every class that we could possibly use in any Java program.

So, with this, we can now write this first example reverse in this polymorphic way by just saying that we take for, reverse takes as argument an array of objects, any kind of object, it does not matter what they are, and then this is just a simple reverse function. All it needs to know is the



length. So, once you have the length, then you start with the 0th element and the (n-1) element and you swap them. So, this is just a swap. It is just swapping  $i$  and  $[(n-1)-i]$ . It is swapping the  $i$ th element from the left with the  $i$ th element from the right.

So, the first element 0 is swapped with (n-1)th element, second element 1 is swapped with (n-2)th element and so on, and you go until the midpoint. So, either you have an odd number of elements, in which case, you swap the two on the other side, or you have an even number of elements, in which case, you go to the midpoint and you swap the two which either side of the midpoint. So that's what this  $i$  from 0 to less than  $(n/2)$  is. So, it is a very straightforward thing.

And all of this, just swapping the position at  $i$  and  $[(n-1)-i]$  does not require me to know anything more about the object except to just move these things around within the array. So, this is easy to implement using this object idea. So, anything that I pass will be compatible with objects. So, that is the idea of this Java class hierarchy because we know that the super type can take a subtype object as a concrete assignment and we use them.

So, if we use find it looks similar. We have seen this one before. We take an array of objects and we look for a specific object  $o$ , but what is kind of hidden under the carpet and which we are now making explicit is that there is something that we need to know about these objects. I need to be able to take the object at position  $i$  in my array and compare it to check whether it is equal to the object I am looking for. So, this “ $=$ ” looks very harmless. But as we just discussed, this may not always happen. There may not be a sensible way to actually compare two objects, but fortunately for us, Java has defined an equals method in the super-class object in this tree root class.

So, therefore, whenever I see “ $=$ ” if I have not made any other kind of sensible way of comparing objects, then by default, it will check for what Java essentially thinks of as objective quality, are they at the same location in memory? So, equal to equal to, in some sense, becomes a trivial check. Is it exactly the same object, same physical object, you might think. I mean, are they actually the same?

Now, of course, in reality, this will never happen. If I give you a box and ask you to check of a similar box and the thing, they are not the same object. You have two copies of the object and you want to check if they are the same. That is the more realistic situation. But then hopefully you have written the method equals which compares them like two dates can be compared, if the day, month and year are the same. But whichever you do, if you do not do anything also the equality always holds. So, just using object is good enough for us.

The third example is sorting. And now we have a problem, because we cannot necessarily assume that two objects can be sorted. So, here our solution was to use an interface. So, the precise requirement that we have for structural polymorphism is to allow something to be done if I have enough capabilities to do it. So, if I want to now use quicksort and an array then I want all the elements in the array to be comparable.

So, comparable now is something which is promised by this interface and it has a function called cmp, which allows me to compare two values by doing a o1.cmp(o2). So, now I can use a[i].cmp(a[j]) and compare it. So, this is more in line with really what we mean by structural polymorphism. We are actually exposing a capability of the object through some limited kind of window and we are ensuring that any object which has that capability is sortable.

(Refer Slide Time: 12:40)



Type consistency

- Polymorphic function to copy an array

```
public static void arraycopy (Object[] src,
                             Object[] tgt){
    int i, limit;
    limit = Math.min(src.length, tgt.length);
    for (i = 0; i < limit; i++){
        tgt[i] = src[i];
    }
}
```



Madhavan Mukund Polymorphism revisited Programming Concepts using Java 4 / 6



## Type consistency

- Polymorphic function to copy an array
- Need to ensure that target array is type compatible with source array
  - Type errors should be flagged at compile time

```
public static void arraycopy (Object[] src,
                             Object[] tgt){
    int i, limit;
    limit = Math.min(src.length, tgt.length);
    for (i = 0; i < limit; i++){
        tgt[i] = src[i];
    }
}
```

*Object*  
*Date Employee*

```
Date[] datearr = new Date[10];
Employee[] emparr = new Employee[10];
arraycopy(datearr, emparr); // Run-time error
```



Madhavan Mukund

Polymorphism revisited

Programming Concepts using Java 4 / 6

## Type consistency

- Polymorphic function to copy an array
- Need to ensure that target array is type compatible with source array
  - Type errors should be flagged at compile time
- More generally source array can be a subtype of the target array

```
public static void arraycopy (Object[] src,
                             Object[] tgt){
    int i, limit;
    limit = Math.min(src.length, tgt.length);
    for (i = 0; i < limit; i++){
        tgt[i] = src[i];
    }
}
```

```
public class Ticket {...}
public class ETicket extends Ticket {...}

Ticket[] tktarr = new Ticket[10];
ETicket[] etktarr = new ETicket[10];
arraycopy(etktarr, tktarr); // Allowed
```



Madhavan Mukund

Polymorphism revisited

Programming Concepts using Java 4 / 6

सिद्धिर्भवति कर्मजा

- Polymorphic function to copy an array
- Need to ensure that target array is type compatible with source array
  - Type errors should be flagged at compile time
- More generally source array can be a subtype of the target array
- But the converse is illegal

```
public static void arraycopy(Object[] src,
                             Object[] tgt){
    int i, limit;
    limit = Math.min(src.length, tgt.length);
    for (i = 0; i < limit; i++){
        tgt[i] = src[i];
    }
}

public class Ticket {...}
public class ETicket extends Ticket {...}

Ticket[] tktarr = new Ticket[10];
ETicket[] etktarr = new ETicket[10];

arraycopy(tktarr, etktarr); // Illegal
```



So, let us look at a slightly more complex example than find. Supposing I want to take one array and copy it into another array, so I have an array copy function which takes a source array and a target array. So, in general, these two may not have the same size. So, what I do is I copy as many as are there in the smaller array. So, I first compute the minimum of the two lengths, the source length and the target length. And now for every  $i$  from 0 to  $(\text{limit}-1)$  I copy from source  $i$  into target  $i$ . So, this now becomes in this Java sense a polymorphic array copy.

The problem is at runtime I might try something like this. I might create an array of dates and of the same length I create an array of employees and now these are two completely incompatible types. Date is got some kind of structure and it is somewhere in the hierarchy and employee is somewhere is. They have no relationship to each other. But both of them because they have this Java hierarchy, date and employee are both compatible with object. So, I can pass either of these arrays the way Java is set it up. If I have a subtype, then an array of subtype is also an array of the super-type.

So, an array of data is compatible with an array of object and array of employees is compatible with an array object. So, I can now say copy the date array into the employee array. And this of course would mean that at an individual level, I will have to take a value which, whose type is declared to be employee and associated with a value, a variable of type employee and associated value of type date, which is not allowed. So, this will give me a runtime error. So, this is a type

error. I am doing in type incorrect assignment. But I am not able to tell, because inside this code, the fact that everything is an object hides the fact that this assignment here could be incompatible. So, I only find out at runtime and this is generally a bad thing. Any type error should be flagged at compile time.

Now, more generally, it is not enough to just say that these must be the same type. It is not necessarily that both are date or both are employee. For instance, we could have one type which is a subtype of another. We could have a class A, which holds information about tickets. And then we could have a subtype called Eticket, which sold something about electronic tickets, which might hold extra information, like what IP address it was booked from and so on.

So, now everywhere where I see a ticket, I can also use an Eticket. So, if I am copying from an Eticket array into a ticket array, it is okay, because every, on the right-hand side, the target value is expecting a ticket and it is getting an Eticket. And everywhere I expect E ticket, I am okay with an Eticket, so this is allowed. On the other hand, if I reverse this and if I try to copy the ticket array into the Eticket array, now this is a problem. Because everywhere I expect an Eticket, if I get something with lesser capabilities, namely the super-type of ticket, this is wrong. So, again, I have to distinguish between these two and somehow capture the fact that there is some constraint on this.

So, in the earlier case the constraint was that these two must actually be equal types. I said I want them both to be date or both to be employee, but more generally, I want that the source type is a subtype of the target type. And that it is not clear how one can do when we use this object kind of notation in order to capture this structural polymorphism.

(Refer Slide Time: 16:30)



## Polymorphic data structures

- Arrays, lists, ... should allow arbitrary elements
- A polymorphic list stores values of type `Object`

```
public class LinkedList{
    private int size;
    private Node first;

    public Object head(){
        Object returnval;
        ...
        return(returnval);
    }

    public void insert(Object newdata){...}

    private class Node {
        private Object data;
        private Node next;
        ...
    }
}
```



Madhavan Mukund

Polymorphism revisited

Programming Concepts using Java 5/6



## Polymorphic data structures

- Arrays, lists, ... should allow arbitrary elements
- A polymorphic list stores values of type `Object`
- Two problems
  - Type information is lost, need casts

```
public class LinkedList{
    private int size;
    private Node first;

    public Object head(){ ... }

    public void insert(Object newdata){...}

    private class Node {...}
}

LinkedList list = new LinkedList();
Ticket t1,t2;

t1 = new Ticket();
list.insert(t1);
t2 = (Ticket)list.head();
// head() returns an Object
```



Madhavan Mukund

Polymorphism revisited

Programming Concepts using Java 5/6

सिद्धिर्भवति कर्मजा



- Arrays, lists, ... should allow arbitrary elements
- A polymorphic list stores values of type `Object`
- Two problems
  - Type information is lost, need casts
  - List need not be homogenous!

```
public class LinkedList{
    private int size;
    private Node first;

    public Object head(){ ... }

    public void insert(Object newdata){...}

    private class Node {...}
}

LinkedList list = new LinkedList();
Ticket t = new Ticket();
Date d = new Date();
list.insert(t);
list.insert(d);
...
```



Related to this is the data structure as a whole, so when we have a data structure as a whole, like an array or a list, what is important is the structure of the data. We are collecting together this data in a sequence. But now whether that sequence contains dates or employees or so on, many of the operations that we perform like reversing and sorting will not depend on the specific objects, but on the capabilities of those objects. So, we should allow arbitrary types to be defined inside a data structure. So, especially, if we have a user defined data sets, like we have a list.

So, then again using this mechanism available to us through objects, so supposing we want a generic linked list which uses a class node to store individual values, then the strategy is to make the data objects stored inside each node of type capital O object, which is this Java route class. And so, whenever we return the first element in this list, it will return something of type object and whenever we add something to the list, it will take something of type object. So, this is similar to our earlier functions like find and reverse and so on.

So, we are using the fact that object is compatible with any value to ensure that this linked list can hold arbitrary values because the nodes inside can hold arbitrary values. So, there are two problems with this. So, the first is that we have a type loss problem. So, supposing I want to actually create a specific list which holds tickets, so I want to create a new list of this type and I create two ticket variables. Remember, we had this ticket class which we talked about. So, I

create t1, which is a new ticket, and I insert it into the list. So, since ticket is compatible with object, this insert will work.

Now, I want to take that object that is there in the list, the head of the list and assign it to t2. So, I want to take this value that head gives and assign it to t2. So, I take list dot head, but unfortunately, if I take list.head(), it comes out as an object. And I have on the left-hand side a variable of type ticket which is more specific than object. It is a subtype of object with everything and subtype of object. But now I cannot expect, I expect something minimally with the capabilities of a ticket. I am not able to handle an object. So, I have to actually cast. So, each time I return this generic object from this list, I have to cast it to the specific type which is a nuisance.

The other more serious problem is that typically when we have a situation like this, we have a kind of sequence of values or an array or anything, we do expect that across the sequence all the values are compatible. So, we want a homogeneous kind of list. So, now, again, we can do this following thing that is we can create a ticket and create a date and insert first the ticket and then the date. So, we can mix up the objects inside this, because everything is going to be compatible with this parameter type object. So, we need not have, at the end of this operation, we cannot enforce in some sense that this list or array has a uniform type. So, again, we are limited.

It is similar to what we said before about the functions which say array copy which wanted to check some compatibility with source array and the target array. So here we want to check some compatibility between one object in the list and another object. We want to make sure that all the objects in the list are actually of the same type.



(Refer Slide Time: 20:12)



## Generic programming in Java

- Java added **generic** programming to address these issues
- Classes and functions can have type parameters
  - `class LinkedList<T>` holds values of type `T`
  - `public T head(){...}` returns a value of same type `T` as enclosing class
- Can describe subclass relationships between type variables
  - `public <S extends T, T> void arraycopy (S[] src, T[] tgt){ ... }`



Mathavan Mahand

Polymorphism revisited

Programming Concepts using Java 6 / 6

So, these are some shortcomings which were there originally in the language, Java. And initially, it was thought that this object-based hierarchy can solve all the problems which one wanted to associate with the structural polymorphism, but these kinds of drawbacks became evident. So, somewhere a few years down the line, Java added something which is called generic programming. So, the idea in generic programming is that you add a new class of syntax called a type parameter. So, for instance, if you want to say that linear list that we had defined can hold an arbitrary value, but that value has a fixed type, then we will say that it is a linear list of type T.

So, we pass a, we associate with this linear list type value T, and so for example, inside that list, if we see a function like head, then it must return an object of the same type as the type that the list was created with. So, there is a way of now matching up what the functions do inside the class with what the class was created with. So, instead of using object everywhere and all objects being independently compatible with any arbitrary type here, you say that I create it with a particular type, then the head will return a value of that same type, insert will expect another value of the same type, and so on. So, we have these type parameters.

And now we also had this problem of different types with some compatibility constraints. So, remember, we said that in an array type, the source and the target could be of two different types. I could have two different type variables S and T. But not every source can be copied to every

target, but I can copy it if S is a subtype of T. So, I can now say that this function involves two variables S and T, two different types, but there is a constraint that S extends T. So, these are the two, these are the ideas that Java introduced in its so-called generics. So, we will look at that next and see exactly how that works.

